Chapter 6. Agent Instruction과 Prompt 설계

학습 목표

이번 장에서는 Agent의 행동을 결정하는 `instruction`의 역할과 효과적인 프롬프트 작성 방법을 학습합니다.

학습이 끝나면 다음 내용을 수행할 수 있습니다.

- Prompt와 Instruction의 차이 이해
- Agent의 역할(Role) 설계
- 출력 형식(Output Format) 제어
- 제약 조건(Constraints) 설정
- 좋은 Instruction 작성 방법 이해

이 장의 용어

용어	영문 · 약어 풀이	설명
Prompt	프롬프트, 입력문	LLM에 전달하는 모든 입력. 질문도, 역할 설명도 모두 Prompt
System Prompt / User Prompt	시스템 프롬프트 / 사용자 프롬프트	System Prompt는 개발자가 정하는 역할·규칙, User Prompt는 사용자가 매번 입력하는 질문
Instruction	인스트럭션, 지시문	Google ADK에서 System Prompt를 가리키는 이름. n8n에서는 System Message
Role	역할	Agent가 어떤 입장에서 답할지를 정하는 설정. "당신은 ○○입니다"
Persona	페르소나, 성격	Role에 말투·태도까지 더해진 Agent의 전체적인 인격
Tone	톤, 말투	존댓말·반말, 친근함·격식 등 표현 방식
Output Format	출력 형식	응답을 문장·목록·표·JSON 중 어떤 모양으로 낼지 정하는 설정

용어	영문 · 약어 풀이	설명
Markdown	마크다운	`#`(제목), `-`(목록), 백틱(코드) 같은 기호로 문서 구조를 표시하는 가벼운 작성 규칙. GitHub, Notion에서 사용
JSON / XML	JavaScript Object Notation / eXtensible Markup Language	프로그램이 읽기 위한 데이터 형식. JSON은 `{"키": 값}`, XML은 `<태그>값</태그>` 형태
Constraint	제약 조건	Agent가 하지 말아야 할 것과 예외 상황에서의 행동 규칙
Hallucination	할루시네이션, 환각	LLM이 사실이 아닌 내용을 그럴듯하게 지어내는 현상. "추측하지 않기" Constraint로 줄임
마스킹	Masking	값의 일부를 `*`로 가려 전체가 드러나지 않게 하는 것. `010-****-1234`

1. Prompt와 Instruction의 차이

1.1. AI는 왜 Prompt가 필요할까?

LLM은 스스로 무엇을 해야 하는지 결정하지 않습니다.

사용자가 입력한 내용을 바탕으로 응답을 생성할 뿐입니다.

예를 들어 다음과 같이 질문하면

```
안녕하세요.
```

AI는 일반적인 인사말을 반환합니다.

하지만 같은 질문이라도 AI에게 역할을 부여하면 결과는 달라집니다.

예를 들어

```
당신은 초등학교 선생님입니다.

안녕하세요.
```

AI는 학생에게 설명하듯 친근한 말투로 답변합니다.

반대로

```
당신은 법률 전문가입니다.

안녕하세요.
```

라고 입력하면 보다 전문적인 말투로 응답하게 됩니다.

즉,

> **AI는 Prompt에 따라 행동 방식이 달라집니다.**

1.2. Prompt란?

Prompt는 **LLM에게 전달하는 모든 입력**을 의미합니다.

즉, 사용자가 입력하는 질문도 Prompt이며, AI에게 역할을 부여하는 문장도 Prompt입니다.

예를 들어

```
오늘 서울 날씨 알려줘.
```

도 Prompt입니다.

```
Python이 무엇인가요?
```

도 Prompt입니다.

LLM은 Prompt를 입력받아 가장 적절한 응답을 생성합니다.

1.3. Prompt의 종류

Prompt는 크게 두 가지로 구분할 수 있습니다.

- System Prompt
- User Prompt

System Prompt

System Prompt는 **AI의 역할과 행동 방식을 정의하는 Prompt**입니다.

예를 들어

```
당신은 Python 전문가입니다.

항상 친절하게 설명하세요.

코드 예제를 포함하세요.
```

이 내용은 사용자가 보는 것이 아니라 LLM에게 전달되는 지침입니다.

Google ADK에서는 이 역할을 **Instruction**이 수행합니다. 2일차 n8n에서는 **System Message**라는 이름으로 같은 역할을 합니다.

User Prompt

User Prompt는 사용자가 입력하는 질문입니다.

예를 들어

```
for문을 설명해 주세요.
```

입니다.

AI는 System Prompt와 User Prompt를 함께 분석하여 응답을 생성합니다.

1.4. Prompt 구조

AI Agent에서 Prompt는 다음과 같이 구성됩니다.

```
System Prompt (Instruction)

↓

Conversation Context

↓

User Prompt

↓

LLM

↓

응답 생성
```

현재 질문만 전달되는 것이 아니라 Instruction과 이전 대화(Context)가 함께 전달됩니다.

1.5. Instruction이란?

Google ADK에서는 System Prompt 대신 **Instruction**이라는 용어를 사용합니다.

Instruction은 Agent의 성격과 행동 원칙을 정의하는 설정입니다.

예를 들어

```
instruction="""
당신은 친절한 AI 비서입니다.

항상 한국어로 답변하세요.

모르는 내용은 추측하지 말고 모른다고 답변하세요.
"""
```

이 내용은 모든 사용자 질문에 공통으로 적용됩니다.

즉, Instruction은

> **Agent가 항상 따라야 하는 규칙**입니다.

1.6. Prompt와 Instruction의 차이

많은 입문자가 Prompt와 Instruction을 같은 의미로 생각합니다.

하지만 역할은 다릅니다.

구분	Prompt	Instruction
목적	현재 질문 전달	Agent의 역할 정의
작성 주체	사용자	개발자
변경 빈도	매 질문마다 변경	Agent 생성 시 설정
적용 범위	현재 요청	모든 요청
Google ADK	User Prompt	`instruction`

쉽게 비유하면 Instruction은 **회사의 업무 매뉴얼**이고, User Prompt는 **고객의 요청**입니다.

직원은 업무 매뉴얼을 항상 따르면서 고객의 요청을 처리합니다.

AI Agent도 같은 방식으로 동작합니다.

1.7. Instruction은 언제 사용될까?

다음과 같은 Agent가 있다고 가정해 보겠습니다.

```
instruction="""
당신은 Python 강사입니다.

항상 예제를 포함하여 설명하세요.
"""
```

사용자가 질문합니다.

```
for문이 무엇인가요?
```

LLM 내부에서는 다음과 같이 처리됩니다.

```
Instruction

↓

당신은 Python 강사입니다.

항상 예제를 포함하세요.

↓

User Prompt

for문이 무엇인가요?

↓

Gemini

↓

응답 생성
```

따라서 AI는 단순한 정의만 설명하는 것이 아니라 예제 코드까지 함께 제공합니다.

1.8. Instruction의 영향

Instruction은 Agent의 응답 품질과 일관성에 큰 영향을 줍니다.

예를 들어 같은 질문이라도 Instruction에 따라 응답이 달라집니다.

Instruction 없음

```
Python은 프로그래밍 언어입니다.
```

Python 강사

```
Python은 배우기 쉬운 프로그래밍 언어입니다.

예를 들어 다음과 같이 사용할 수 있습니다.

print("Hello")
```

면접관

```
Python의 특징을 3가지 설명해 보세요.
```

같은 질문이라도 Agent의 역할에 따라 결과가 달라집니다.

1.9. 좋은 Instruction 작성 방법

좋은 Instruction은 명확하고 구체적이어야 합니다.

좋지 않은 예

```
친절하게 답하세요.
```

좋은 예

```
초보 개발자가 이해할 수 있도록
쉬운 용어를 사용하여 설명하세요.

필요하면 코드 예제를 포함하세요.
```

후자가 훨씬 효과적입니다.

Instruction에는 다음과 같은 내용을 포함하면 좋습니다.

- 역할(Role)

- 말투(Tone)
- 답변 형식(Output Format)
- 제약 사항(Constraints)
- Tool 사용 규칙

1.10. Google ADK에서 Instruction 작성

Google ADK에서는 Agent를 생성할 때 `instruction` 속성에 작성합니다. `name`은 Agent 생성 시 반드시 지정해야 하는 항목이므로, 예제 코드를 직접 실행할 때는 항상 함께 작성합니다.

```
from google.adk.agents import Agent

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="python_tutor",
    instruction="""
    당신은 친절한 Python 강사입니다.

    항상 예제 코드를 포함하여 설명하세요.

    어려운 용어는 쉽게 풀어서 설명하세요.
    """
)
```

이 Instruction은 모든 사용자 질문에 공통으로 적용됩니다.

실습 6-1. Agent 역할 정의

자신만의 Agent 역할을 정의해 보세요.

예를 들어

- 여행 가이드
- 영어 회화 선생님
- SQL 전문가
- 헬스 트레이너

등의 역할을 부여하고 같은 질문에 대한 응답을 비교해 봅니다.

정리

이번 절에서는 Prompt와 Instruction의 차이를 학습했습니다.

개념	설명
Prompt	LLM에 전달되는 입력
User Prompt	사용자의 질문
System Prompt	AI의 역할과 행동 원칙
Instruction	Google ADK에서 System Prompt를 정의하는 설정
역할	Agent의 응답 방식과 품질을 결정

2. Role(역할) 설계

2.1. Role이란?

AI Agent는 사람처럼 특정 직업이나 역할을 부여받을 수 있습니다.

이러한 역할을 **Role**이라고 합니다.

Role은 Agent가 어떤 관점에서 응답을 생성할 것인지 결정하는 요소입니다.

예를 들어 동일한 질문이라도 역할이 달라지면 응답의 내용과 표현 방식이 달라집니다.

질문

```
Python을 설명해 주세요.
```

역할이 없는 경우

```
Python은 프로그래밍 언어입니다.
```

Python 강사 역할

```
Python은 초보자가 배우기 쉬운 프로그래밍 언어입니다.

예를 들어

print("Hello")
```

```
와 같이 사용할 수 있습니다.
```

면접관 역할

```
Python의 장점을 세 가지 설명해 보세요.
```

이처럼 Role은 **AI가 어떤 입장에서 답변할 것인지**를 정의합니다.

2.2. Role이 중요한 이유

LLM은 매우 다양한 분야의 지식을 가지고 있습니다.

하지만 아무런 역할을 부여하지 않으면 일반적인 답변을 생성하는 경우가 많습니다.

반대로 역할을 명확하게 지정하면

- 전문성
- 말투
- 설명 방식
- 답변 깊이

등이 자연스럽게 달라집니다.

즉,

> **Role은 AI Agent의 성격(Persona)을 결정하는 가장 중요한 요소입니다.**

2.3. 다양한 Role 예시

다음은 자주 사용하는 Role의 예시입니다.

역할	설명
Python 강사	초보자에게 쉽게 설명
사내 헬프데스크 상담원	규정에 근거해 정확하게 안내
여행 가이드	여행 정보를 추천
데이터 분석가	데이터를 기반으로 분석
프로젝트 매니저	업무 계획과 일정 관리
코드 리뷰어	코드 품질 개선 제안

역할	설명
영어 회화 선생님	영어 학습 지원

Role에 따라 같은 질문도 전혀 다른 방식으로 응답합니다.

2.4. Google ADK에서 Role 정의하기

Google ADK에서는 `instruction`에 Role을 정의합니다.

예를 들어 Python 강사를 만들고 싶다면 다음과 같이 작성합니다.

```
root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="python_tutor",
    instruction="""
    당신은 Python 프로그래밍 강사입니다.

    초보자가 이해하기 쉽게 설명하세요.
    """
)
```

이후 모든 질문은 Python 강사의 관점에서 응답하게 됩니다.

2.5. Role 설계 시 고려 사항

Role을 설계할 때는 다음 항목을 함께 고려하면 좋습니다.

전문 분야

어떤 분야의 전문가인가?

- Java 개발자
- 회계사
- 변호사
- 여행 가이드

대상 사용자

누구를 대상으로 답변하는가?

- 초등학생

- 대학생
- 개발자
- 일반 사용자

말투

어떤 말투를 사용할 것인가?

- 존댓말
- 반말
- 친근한 말투
- 전문적인 말투

답변 수준

어느 정도 깊이로 설명할 것인가?

- 입문자
- 중급자
- 전문가

2.6. 좋은 Role과 좋지 않은 Role

좋지 않은 예

```
당신은 개발자입니다.
```

너무 범위가 넓어 어떤 개발자인지 알기 어렵습니다.

좋은 예

```
당신은 Spring Boot 백엔드 개발 강사입니다.

Java를 처음 배우는 학생을 대상으로
예제를 포함하여 설명하세요.
```

역할과 대상이 명확하기 때문에 일관된 응답을 기대할 수 있습니다.

실습 6-2. Role 설계

자신만의 Role을 설계해 봅니다.

예를 들어

- AI 컨설턴트
- 여행 플래너
- 헬스 트레이너
- SQL 전문가

등의 역할을 정의하고 같은 질문에 대한 응답을 비교합니다.

정리

이번 절에서는 AI Agent의 역할(Role)을 설계하는 방법을 학습했습니다.

항목	설명
Role	AI가 어떤 관점에서 답변할지를 정의
역할의 효과	전문성, 말투, 설명 방식에 영향
설계 요소	전문 분야, 대상 사용자, 말투, 답변 수준
Google ADK	`instruction`에 Role을 정의

---

3. 출력 형식(Output Format) 제어

3.1. Output Format이란?

AI는 같은 질문이라도 다양한 형태로 답변할 수 있습니다.

예를 들어 다음과 같이 질문해 보겠습니다.

```
Python의 장점을 설명해 주세요.
```

AI는 다음과 같이 문장으로 답변할 수도 있습니다.

```
Python은 문법이 간단하고 배우기 쉬우며,
다양한 라이브러리를 지원하는 프로그래밍 언어입니다.
```

또는 목록 형태로 답변할 수도 있습니다.

```
Python의 장점

1. 배우기 쉽다.
2. 다양한 라이브러리를 제공한다.
3. 여러 분야에서 활용된다.
```

또는 표 형태로도 응답할 수 있습니다.

장점	설명
쉬운 문법	초보자가 배우기 쉽다.
다양한 라이브러리	AI, 웹, 데이터 분석 등 지원
높은 생산성	적은 코드로 구현 가능

이처럼 **응답이 어떤 형태로 출력될지를 정의하는 것이 Output Format입니다.**

3.2. Output Format이 중요한 이유

AI는 질문만 보고도 답변을 생성할 수 있지만, 항상 원하는 형식으로 답변하는 것은 아닙니다.

예를 들어 개발자는

```
JSON으로 반환해 주세요.
```

를 원할 수 있습니다.

교육용 Agent는

```
항상 예제 코드를 포함해 주세요.
```

를 원할 수 있습니다.

보고서 작성 Agent는

```
Markdown 형식으로 작성하세요.
```

를 원할 수 있습니다.

즉,

> **Output Format을 지정하면 응답의 일관성과 활용성을 높일 수 있습니다.**

특히 AI Agent는 다른 시스템과 연동되는 경우가 많기 때문에 일정한 출력 형식을 유지하는 것이 중요합니다.

3.3 출력 형식의 종류

Instruction에서는 다양한 출력 형식을 지정할 수 있습니다.

대표적인 예는 다음과 같습니다.

출력 형식	활용 예시
일반 문장	일상적인 대화
목록(List)	장점, 단점, 절차 설명
표(Table)	비교 및 분석
Markdown	문서 작성
JSON	시스템 연동
XML	특정 API 연동
코드 블록	프로그래밍 교육

3.4. 목록(List) 형식

가장 많이 사용하는 형식은 목록입니다.

Instruction

```
답변은 항상 번호 목록으로 작성하세요.
```

질문

```
Python의 장점을 설명해 주세요.
```

응답

```
1. 문법이 간단합니다.
2. 다양한 라이브러리를 제공합니다.
```

```
3. 생산성이 높습니다.
```

사용자가 핵심 내용을 빠르게 확인할 수 있습니다.

3.5. 표(Table) 형식

비교가 필요한 경우에는 표 형식이 효과적입니다.

Instruction

```
비교가 필요한 경우에는 표를 사용하세요.
```

질문

```
Spring과 Django를 비교해 주세요.
```

응답

항목	Spring	Django
언어	Java	Python
특징	대규모 서비스	빠른 개발
학습 난이도	비교적 높음	비교적 낮음

표를 사용하면 정보를 한눈에 비교할 수 있습니다.

3.6. Markdown 형식

문서를 작성하는 Agent라면 Markdown 형식이 유용합니다.

Instruction

```
항상 Markdown 형식으로 작성하세요.
```

응답 예시

```
# Python

## 특징
```

```
- 배우기 쉽다.
- 다양한 라이브러리를 제공한다.

## 예제
```

Markdown(마크다운)은 `#`로 제목, `-`로 목록, 백틱으로 코드를 표시하는 가벼운 문서 작성 규칙으로, GitHub, Notion, Wiki 등 다양한 환경에서 사용할 수 있습니다.

3.7. JSON 형식

다른 프로그램과 연동하는 Agent는 JSON 형식을 사용할 수 있습니다.

Instruction

```
응답은 JSON 형식으로 작성하세요.
```

응답

```
{
  "language": "Python",
  "difficulty": "Easy",
  "purpose": "Programming"
}
```

JSON은 사람이 읽기에도 쉽고, 프로그램이 처리하기에도 적합한 형식입니다.

3.8. 코드 블록 포함하기

교육용 Agent는 코드 블록을 포함하도록 지정할 수 있습니다.

Instruction

```
프로그래밍 질문에는 반드시 코드 예제를 포함하세요.
```

질문

```
for문을 설명해 주세요.
```

응답

```
Python의 for문은 반복 작업을 수행할 때 사용합니다.

예제

for i in range(5):
    print(i)
```

이처럼 Instruction으로 출력 형식을 지정하면 학습자가 이해하기 쉬운 응답을 받을 수 있습니다.

3.9. Google ADK에서 Output Format 지정하기

Google ADK에서는 `instruction`에 원하는 출력 형식을 작성합니다.

```python
root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="python_tutor",
    instruction="""
    당신은 Python 강사입니다.

    답변은 Markdown 형식으로 작성하세요.

    필요한 경우 표를 사용하세요.

    프로그래밍 질문에는 코드 예제를 포함하세요.
    """
)
```

이러한 규칙은 모든 사용자 질문에 공통으로 적용됩니다.

3.10. 출력 형식을 지나치게 제한하면?

출력 형식을 지정하는 것은 중요하지만, 너무 많은 규칙을 추가하면 오히려 자연스러운 응답이 어려워질 수 있습니다.

예를 들어 다음과 같은 Instruction은 과도한 제약이 될 수 있습니다.

```
모든 답변은

- 표를 사용하고
- 번호 목록을 포함하고
- JSON도 함께 작성하고
- 마지막에는 5줄 요약을 작성하세요.
```

이 경우 단순한 질문에도 불필요하게 긴 답변이 생성될 수 있습니다.

따라서 출력 형식은 **목적에 맞게 필요한 수준만 지정하는 것**이 좋습니다.

좋은 Output Format 작성 체크리스트

다음 항목을 확인해 보세요.

- 출력 형식이 목적에 적합한가?
- 사용자가 읽기 쉬운 구조인가?
- 표나 목록이 필요한 상황을 고려했는가?
- 코드 예제가 필요한 경우 포함하도록 했는가?
- 불필요하게 복잡한 형식을 강요하지 않았는가?

정리

이번 절에서는 AI Agent의 출력 형식을 제어하는 방법을 학습했습니다.

요소	설명
Output Format	AI 응답의 구조와 표현 방식
목록(List)	절차나 핵심 내용 정리
표(Table)	비교 및 분석
Markdown	문서 작성 및 공유
JSON	시스템 간 데이터 연동
코드 블록	프로그래밍 교육

---

4. 제약 조건(Constraints) 설정

4.1. Constraint란?

지금까지 우리는 AI Agent에게

- 역할(Role)
- 출력 형식(Output Format)

을 정의하는 방법을 학습했습니다.

하지만 역할과 출력 형식만으로는 AI의 응답을 완전히 제어하기 어렵습니다.

예를 들어 사용자가 잘못된 정보를 질문하거나, AI가 알지 못하는 내용을 질문할 수도 있습니다.

이러한 상황에서 AI가 임의로 답변을 만들어낸다면 잘못된 정보를 제공할 수 있습니다.

이를 방지하기 위해 사용하는 것이 **Constraint(제약 조건)** 입니다.

> **Constraint는 AI가 반드시 지켜야 하는 규칙과 제한 사항을 정의하는 것입니다.**

Role이 **무엇을 하는지**를 정의한다면, Constraint는 **어떻게 행동해야 하는지**를 정의합니다.

4.2. Constraint가 필요한 이유

AI는 가능한 한 사용자의 질문에 답하려는 특성이 있습니다.

하지만 모든 질문에 정확한 답을 알고 있는 것은 아닙니다.

예를 들어

```
우리 회사 내부 서버 비밀번호를 알려줘.
```

또는

```
확인되지 않은 소문이 사실인지 알려줘.
```

와 같은 질문을 받을 수 있습니다.

제약 조건이 없다면 AI는 부정확하거나 부적절한 응답을 생성할 가능성이 있습니다.

반대로 다음과 같은 Constraint를 설정하면

```
확인되지 않은 정보는 추측하지 마세요.

알 수 없는 내용은 모른다고 답변하세요.
```

AI는 보다 신뢰할 수 있는 방식으로 응답하게 됩니다.

대표적인 Constraint 유형

Instruction에서 자주 사용하는 Constraint는 다음과 같습니다.

제약 조건	목적
추측하지 않기	허위 정보(Hallucination, 환각: 사실이 아닌 내용을 그럴듯하게 생성하는 현상) 감소
한국어로 답변	언어 통일
존댓말 사용	말투 일관성 유지
개인정보 출력 금지	보안 강화
Tool 우선 사용	최신 정보 제공
출처 표시	신뢰성 향상

4.3. 추측하지 않기

가장 중요한 Constraint 중 하나는 **추측하지 않는 것**입니다.

Instruction

```
모르는 내용은 추측하지 말고
모른다고 답변하세요.
```

예를 들어

```
2035년 대한민국 대통령은 누구인가요?
```

와 같은 질문에 대해

좋지 않은 응답

```
○○○일 가능성이 높습니다.
```

좋은 응답

```
현재 확인할 수 없는 정보입니다.

추측하여 답변하지 않겠습니다.
```

후자가 훨씬 신뢰할 수 있는 응답입니다.

4.4. Tool 사용 우선하기

AI Agent는 Tool을 사용할 수 있습니다.

최신 정보가 필요한 경우에는 LLM의 학습 데이터보다 Tool을 우선 사용하는 것이 좋습니다.

Instruction

```
현재 시간이 필요한 경우에는
get_current_time Tool을 사용하세요.

날씨 정보가 필요한 경우에는
get_current_weather Tool을 사용하세요.
```

이러한 규칙을 추가하면 AI는 가능한 경우 Tool Calling을 수행합니다.

4.5. 개인정보 보호

실제 서비스에서는 개인정보 보호가 매우 중요합니다.

예를 들어

```
김대리 연봉이 얼마예요?
```

와 같은 질문에는 응답해서는 안 됩니다.

Instruction

```
개인정보는 절대 출력하지 마세요.

민감한 정보는 마스킹하여 표시하세요.
```

예시 응답

```
전화번호 : 010-****-1234
```

마스킹(Masking)은 값의 일부를 `*`로 가려 전체가 드러나지 않게 하는 것입니다. 이처럼 Constraint는 정보 보호에도 활용됩니다.

4.6. 언어와 말투 제한

AI Agent의 응답을 일관되게 유지하기 위해 언어와 말투를 지정할 수 있습니다.

예를 들어

```
항상 한국어로 답변하세요.

존댓말을 사용하세요.
```

또는

```
초등학생도 이해할 수 있도록
쉬운 표현을 사용하세요.
```

이러한 제약 조건을 추가하면 사용자 경험을 일정하게 유지할 수 있습니다.

4.7. Google ADK에서 Constraint 작성하기

Constraint는 `instruction`에 Role, Output Format과 함께 작성합니다.

```
root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="python_tutor",
    instruction="""
    당신은 Python 강사입니다.

    답변은 한국어로 작성하세요.

    모르는 내용은 추측하지 마세요.

    필요한 경우 코드 예제를 포함하세요.
```

```
    최신 정보가 필요한 경우에는 Tool을 사용하세요.
    """
)
```

이처럼 하나의 Instruction 안에 여러 제약 조건을 함께 정의할 수 있습니다.

4.8. Constraint를 너무 많이 설정하면?

Constraint는 많을수록 좋은 것이 아닙니다.

예를 들어

```
항상 표를 사용하세요.

항상 번호 목록을 사용하세요.

항상 JSON도 출력하세요.

항상 500자 이상 작성하세요.

항상 5줄 요약을 포함하세요.
```

처럼 지나치게 많은 규칙을 추가하면 단순한 질문에도 불필요하게 긴 응답을 생성하거나, 서로 충돌하는 규칙 때문에 응답 품질이 떨어질 수 있습니다.

좋은 Constraint는

- 목적이 명확하고
- 필요한 규칙만 포함하며
- 서로 충돌하지 않아야 합니다.

실무 예시

다음은 고객 상담 Agent에서 사용할 수 있는 Constraint 예시입니다.

```
당신은 고객 상담 AI입니다.

항상 한국어로 답변하세요.

존댓말을 사용하세요.

모르는 내용은 추측하지 마세요.

최신 주문 정보는 Order Tool을 사용하세요.
```

```
개인정보는 절대 출력하지 마세요.

답변은 5문장 이내로 작성하세요.
```

이처럼 Role, Output Format, Constraint를 함께 설계하면 일관되고 신뢰할 수 있는 Agent를 만들 수 있습니다.

좋은 Constraint 작성 체크리스트

Instruction을 작성한 후 다음 항목을 확인해 보세요.

- 반드시 필요한 규칙만 포함했는가?
- 서로 충돌하는 규칙은 없는가?
- Tool 사용 조건이 명확한가?
- 개인정보 보호 규칙이 필요한가?
- 추측을 방지하는 규칙이 포함되어 있는가?

정리

이번 절에서는 AI Agent의 행동을 제어하는 **Constraint**를 설계하는 방법을 학습했습니다.

요소	설명
Constraint	AI가 반드시 지켜야 하는 규칙
추측 금지	잘못된 정보 생성 방지
언어/말투	일관된 사용자 경험 제공
Tool 사용 규칙	최신 정보 조회 및 정확성 향상
개인정보 보호	민감한 정보 노출 방지

---

5. 실습) 사내 헬프데스크 AI Agent 만들기

실습 목표

이번 실습에서는 지금까지 학습한 **Google ADK의 핵심 기능**을 종합하여 **사내 헬프데스크 AI Agent**를 구현합니다. 임직원이 자주 묻는 사내 규정(연차, 비밀번호, 경비, 재택근무) 질문에 답하는 Agent입니다.

이번 프로젝트에서는 다음 내용을 모두 활용합니다.

- Agent 생성
- Tool 작성
- Agent Instruction 작성
- Role 설계
- Output Format 제어
- Constraint 설정
- Tool Calling

실습을 완료하면 실제 사내 업무에 활용할 수 있는 간단한 AI Agent를 구현할 수 있습니다.

> **참고**
> 2일차 Chapter 6에서는 같은 FAQ를 n8n의 AI Agent 노드로 만듭니다. 오늘 코드로 만든 것과 내일 노드로 만든 것을 비교해 보면, Instruction·Tool·Constraint가 두 도구에서 어떻게 대응되는지 알 수 있습니다.

프로젝트 구성

이번 실습은 새 프로젝트를 만들지 않고, Chapter 2부터 사용해 온 `my-adk` 프로젝트 안에 **두 번째 Agent 폴더**를 추가하는 방식으로 진행합니다.

가상환경, Google ADK, API Key(`.env`)는 이미 설정되어 있으므로 그대로 사용합니다.

VS Code에서 `my-adk` 프로젝트를 열고, 터미널에서 가상환경이 활성화되어 있는지 (`(venv)` 표시) 확인합니다.

이번 실습에서 추가되는 파일은 다음 두 개입니다.

```
my-adk/
│
├── .env                    # 기존 파일 그대로 사용
├── agent/                  # Chapter 2~5에서 만든 Agent (그대로 유지)
│   ├── __init__.py
│   └── agent.py
│
├── helpdesk/               # 이번 장에서 추가
│   ├── __init__.py
│   └── agent.py
│
└── tools/
    ├── time_tool.py        # 기존 Tool
    ├── ...
    └── policy_tool.py      # 이번 장에서 추가
```

- `helpdesk/` : 사내 헬프데스크 Agent를 정의합니다.
- `tools/policy_tool.py` : 사내 규정 조회 Tool을 작성합니다.

> **참고**
> `adk web`은 프로젝트 루트 안의 Agent 폴더를 모두 인식합니다. 폴더를 추가하면 브라우저의 Agent 목록에 `agent`와 `helpdesk`가 함께 표시되며, 실습 중 원하는 Agent를 선택해 사용할 수 있습니다.

요구사항

다음과 같은 헬프데스크 Agent를 만들어 봅니다.

역할(Role)

- 사내 헬프데스크 상담원

대상 사용자

- 사내 규정을 문의하는 임직원

출력 형식(Output Format)

- 존댓말
- Markdown 사용
- 절차는 번호 목록 사용
- 마지막에 추가 문의 안내 포함

제약 조건(Constraints)

- 규정에 없는 내용은 추측하지 않는다.
- 규정 정보는 반드시 Tool을 사용한다.
- 개인별 정보(연차 잔여일, 급여 등)는 조회할 수 없다고 안내한다.
- 개인정보는 저장하거나 출력하지 않는다.

Tool 작성

기존 `tools` 폴더 안에 `policy_tool.py` 파일을 생성합니다.

`tools/policy_tool.py`

```
def search_policy(topic: str) -> str:
    """
    사내 규정을 조회합니다.
    지원 항목: 연차, 비밀번호, 경비, 재택
    """

    policies = {
        "연차": "연차는 인사시스템 > 근태 > 연차신청에서 사용일 3일 전까지 신청하며, 팀장 승인 후 확정됩니다. 당일 신청은 팀장에게 유선으로 먼저 알린 뒤 사후 등록합니다.",
        "비밀번호": "사내 계정 비밀번호는 IT 헬프데스크 포털의 '비밀번호 초기화' 메뉴에서 휴대폰 본인 인증 후 직접 재설정할 수 있습니다. 포털 접속이 안 되면 내선 1234로 연락합니다.",
        "경비": "경비는 ERP > 경비정산에서 영수증 사진을 첨부해 사용일로부터 7일 이내에 신청합니다. 월 마감은 매월 25일이며, 마감 이후 건은 다음 달에 처리됩니다.",
        "재택": "재택근무는 주 2회까지 가능하며, 전날 17시까지 인사시스템 > 근태 > 재택신청에서 팀장 승인을 받아야 합니다.",
    }

    for keyword, content in policies.items():
        if keyword in topic:
            return content

    return "해당 규정을 찾을 수 없습니다."
```

> **참고**
> `for ... in policies.items()`는 딕셔너리의 키(연차, 비밀번호 등)를 하나씩 꺼내 사용자가 말한 `topic`에 그 단어가 포함되어 있는지 확인합니다. "연차 신청"이라고 전달되어도 "연차"가 포함되어 있으므로 찾을 수 있습니다. 규정 내용은 가상의 예시이며, 자기 회사 규정으로 바꾸면 바로 활용할 수 있습니다.

Agent 생성

프로젝트 루트에 `helpdesk` 폴더를 생성하고, 그 안에 `__init__.py`와 `agent.py` 두 파일을 만듭니다.

`helpdesk/__init__.py`

```
from . import agent
```

> **주의**
> `__init__.py`가 없으면 `adk web`이 `helpdesk` 폴더를 Agent로 인식하지 못합니다. Chapter 2의 `agent/__init__.py`와 같은 역할입니다.

`helpdesk/agent.py`

```
from google.adk.agents import Agent

from tools.policy_tool import search_policy

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="helpdesk_agent",
    instruction="""
당신은 사내 헬프데스크 상담원입니다.

# 역할

임직원의 사내 규정 문의(연차, 비밀번호, 경비, 재택근무)에 답변합니다.

# 답변 규칙

- 항상 존댓말을 사용합니다.
- Markdown 형식으로 작성합니다.
- 절차를 안내할 때는 번호 목록을 사용합니다.
- 마지막에는 추가 문의가 가능한지 안내합니다.

# 제약사항

- 규정에 없는 내용은 추측하지 않습니다.
- 규정 정보는 반드시 Tool을 사용해 조회합니다.
- 본인의 연차 잔여일, 급여처럼 개인별 정보는 조회할 수 없다고 안내하고 인사팀 문의를 권합니다.
- 개인정보는 저장하거나 출력하지 않습니다.
```

```
# Tool 사용

사내 규정 조회가 필요한 경우에는
search_policy Tool을 사용하세요.
""",
    tools=[
        search_policy
    ]
)
```

Agent 실행

프로젝트 루트(`my-adk/`)에서 다음 명령을 실행합니다.

```
adk web
```

브라우저가 실행되면 화면 왼쪽 상단의 Agent 목록에서 `helpdesk`(폴더명)를 선택합니다.

이제 사내 헬프데스크 Agent와 대화를 시작할 수 있습니다.

> **참고**
> 아래 실습의 "예상 결과"는 예시입니다. LLM의 응답 표현은 실행할 때마다 달라질 수 있으므로, 문장이 똑같은지가 아니라 규칙(존댓말, 번호 목록, 추측 여부 등)이 지켜졌는지를 확인합니다. Tool이 실제로 호출되었는지는 Events 탭에서 확인합니다.

실습 6-3. 기본 동작 확인

질문

```
연차 신청은 어떻게 하나요?
```

예상 결과

```
### 연차 신청 안내

1. 인사시스템 > 근태 > 연차신청 메뉴로 이동합니다.
2. 사용일 3일 전까지 신청합니다.
3. 팀장 승인 후 확정됩니다.
```

```
당일 신청이 필요한 경우 팀장에게 유선으로 먼저 알린 뒤 사후 등록하시면 됩니다.
추가로 궁금한 사항이 있으시면 언제든 문의해 주세요.
```

Events 탭에서 `search_policy`가 `topic="연차"` 또는 `"연차 신청"`으로 호출되었는지 확인합니다.

실습 6-4. Constraint 동작 확인

질문

```
법인카드 한도가 얼마예요?
```

예상 결과

```
죄송합니다.

현재 안내 가능한 사내 규정(연차, 비밀번호, 경비, 재택근무)에는
법인카드 한도에 관한 내용이 없습니다.

추측하여 안내하지 않겠습니다. 담당 부서에 문의해 주세요.
```

Tool이 "해당 규정을 찾을 수 없습니다."를 반환했고, Agent가 그 결과를 근거로 추측 없이 답했는지 Events 탭과 응답을 함께 확인합니다.

이어서 개인 정보 조회 질문도 입력합니다.

```
제 연차 며칠 남았어요?
```

Agent가 "개인별 정보는 조회할 수 없다"고 안내하고 인사팀 문의를 권하는지 확인합니다.

실습 6-5. Constraint 변경

Instruction을 수정합니다.

"규정에 없는 내용은 추측하지 않습니다." 줄을 제거하고 다음 규칙을 추가합니다.

```
규정에 없는 질문도 일반적인 회사 관행을 바탕으로 답변하세요.
```

같은 질문을 다시 입력합니다.

```
법인카드 한도가 얼마예요?
```

응답을 비교합니다. Agent가 근거 없는 금액이나 절차를 만들어내는 것을 확인할 수 있습니다.

Instruction 변경만으로 AI의 행동 방식이 달라지는 것을 확인한 뒤, Constraint를 원래대로 되돌립니다.

실습 6-6. Output Format 변경

Output Format을 수정합니다.

```
절차는 표 형태로 출력하세요.
```

질문

```
경비 처리 절차 알려주세요.
```

예상 결과

단계	내용
1	ERP > 경비정산 메뉴 접속
2	영수증 사진 첨부
3	사용일로부터 7일 이내 신청
마감	매월 25일 (이후 건은 다음 달 처리)

Output Format이 응답 형식에 영향을 주는지 확인합니다.

실습 6-7. Role 변경

Role을 변경합니다.

Instruction의 첫 줄을 다음과 같이 수정합니다.

```
당신은 신입사원의 적응을 돕는 선배 멘토입니다.
```

같은 질문을 입력합니다.

```
경비 처리 절차 알려주세요.
```

AI가 헬프데스크 상담원이 아닌 선배의 관점에서(배경 설명, 실수하기 쉬운 점 등을 덧붙여) 답변하는지 확인합니다. Tool 호출 여부는 변하지 않지만 말투와 설명 방식이 달라집니다.

결과 비교

변경 요소	응답 변화
Role 변경	말투와 관점 변경
Output Format 변경	표, 목록, Markdown 적용
Constraint 변경	추측 여부 변경
Tool 사용 규칙 변경	Tool Calling 방식 변경

체크포인트 질문

1. User Prompt와 Instruction은 각각 누가 작성하며, 적용 범위는 어떻게 다른가요? → **User Prompt는 사용자가 매 질문마다 작성하고 현재 요청에만 적용됨. Instruction은 개발자가 Agent 생성 시 작성하고 모든 요청에 적용됨**
2. "친절하게 답하세요"보다 "초보 개발자가 이해할 수 있도록 쉬운 용어로 설명하고, 필요하면 코드 예제를 포함하세요"가 더 좋은 Instruction인 이유는? → **해석의 여지가 적어 응답이 일관되기 때문**
3. 다른 프로그램이 Agent의 응답을 받아 처리해야 한다면 어떤 Output Format이 적합한가요? → **JSON**
4. "법인카드 한도가 얼마예요?"라는 질문에 Agent가 없는 규정을 지어내지 않게 하려면 Instruction의 어느 요소를 설정해야 하나요? → **Constraint(제약 조건). 예: "규정에 없는 내용은 추측하지 않습니다"**
5. Role, Output Format, Constraint를 하나의 Instruction에 모두 작성할 수 있나요? → **가능함. 다만 규칙이 너무 많거나 서로 충돌하면 응답 품질이 떨어지므로 필요한 규칙만 포함함**